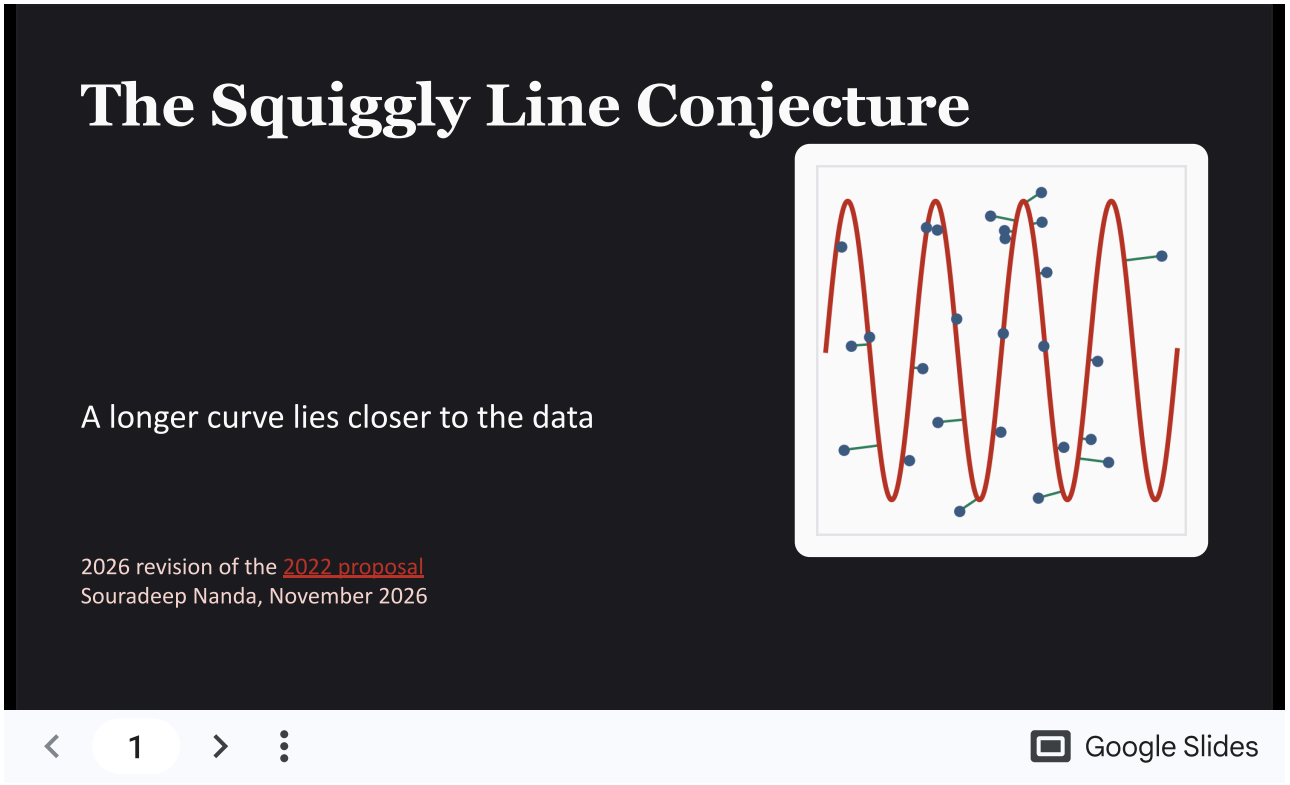
The Squiggly Line Conjecture: An Idea From 2022 and What the Test Found

10 Nov 2026

A picture that was meant to tie together the lottery ticket hypothesis, double descent and faster learning in large networks. The geometry turned out to be true. The learning claim did not.

In 2022, the year Stable Diffusion was released, I had an idea about why large networks learn fast. In 2025 I put it into a slide deck and wrote the first scripts. In 2026 I tested it properly and revised the deck. This post is the written version of the 2026 slides.



The short version: a squigglier curve really is closer to random points, but only in a distance that training does not use. So the idea does not explain why large networks learn fast. The link to the lottery ticket hypothesis and to double descent was always a claim in the deck, and I did not derive either effect. Below are the idea, my experiments, a correction to my derivation, and what published work says.

The claim of the original deck

The deck connects three puzzles of large networks in one geometric picture:

- The lottery ticket hypothesis: a large random network contains a small subnetwork that trains well.
- Double descent: test error falls, rises, then falls again as the model grows.
- Why larger networks learn faster.

The deck states this link as a claim. It does not derive the three effects.

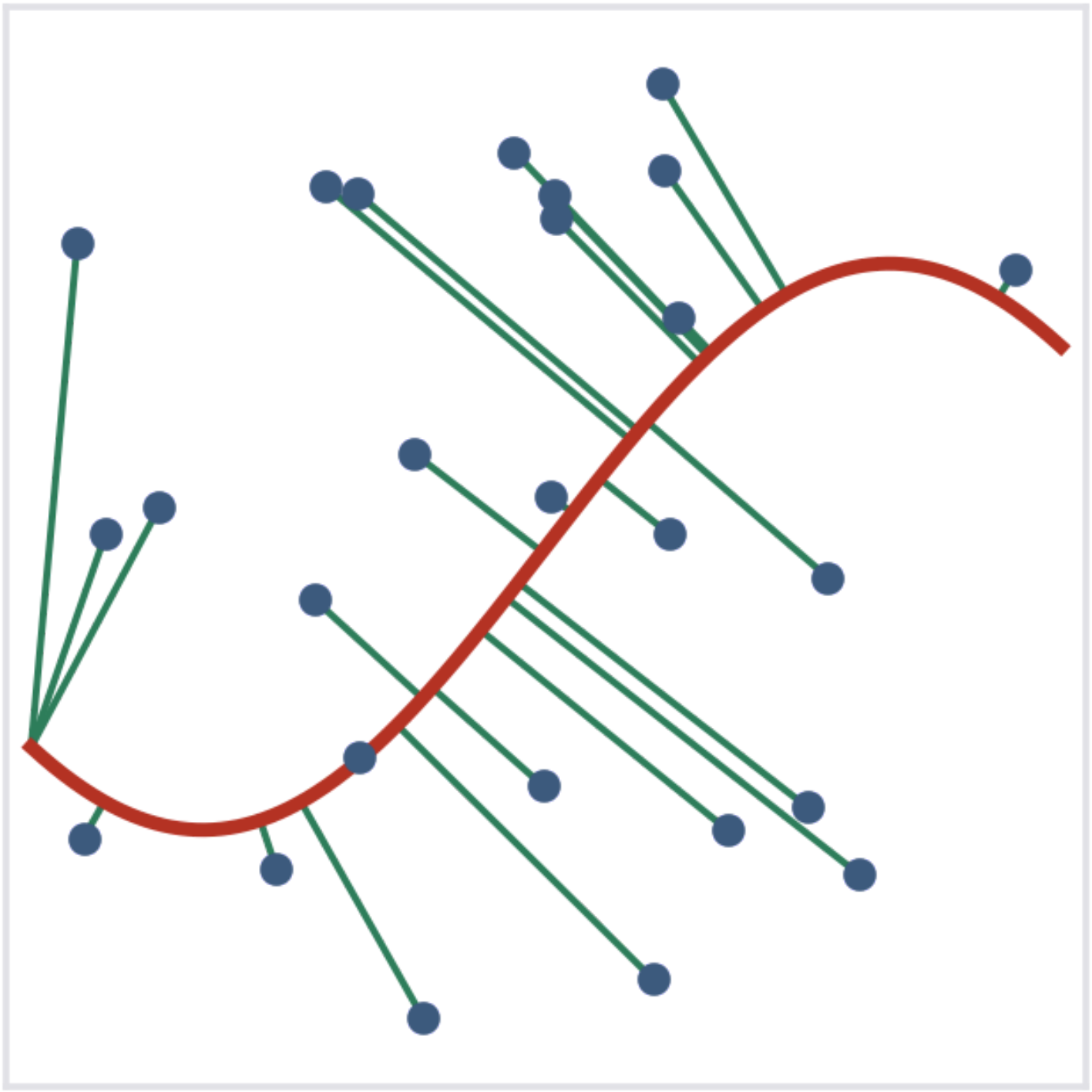
The core idea

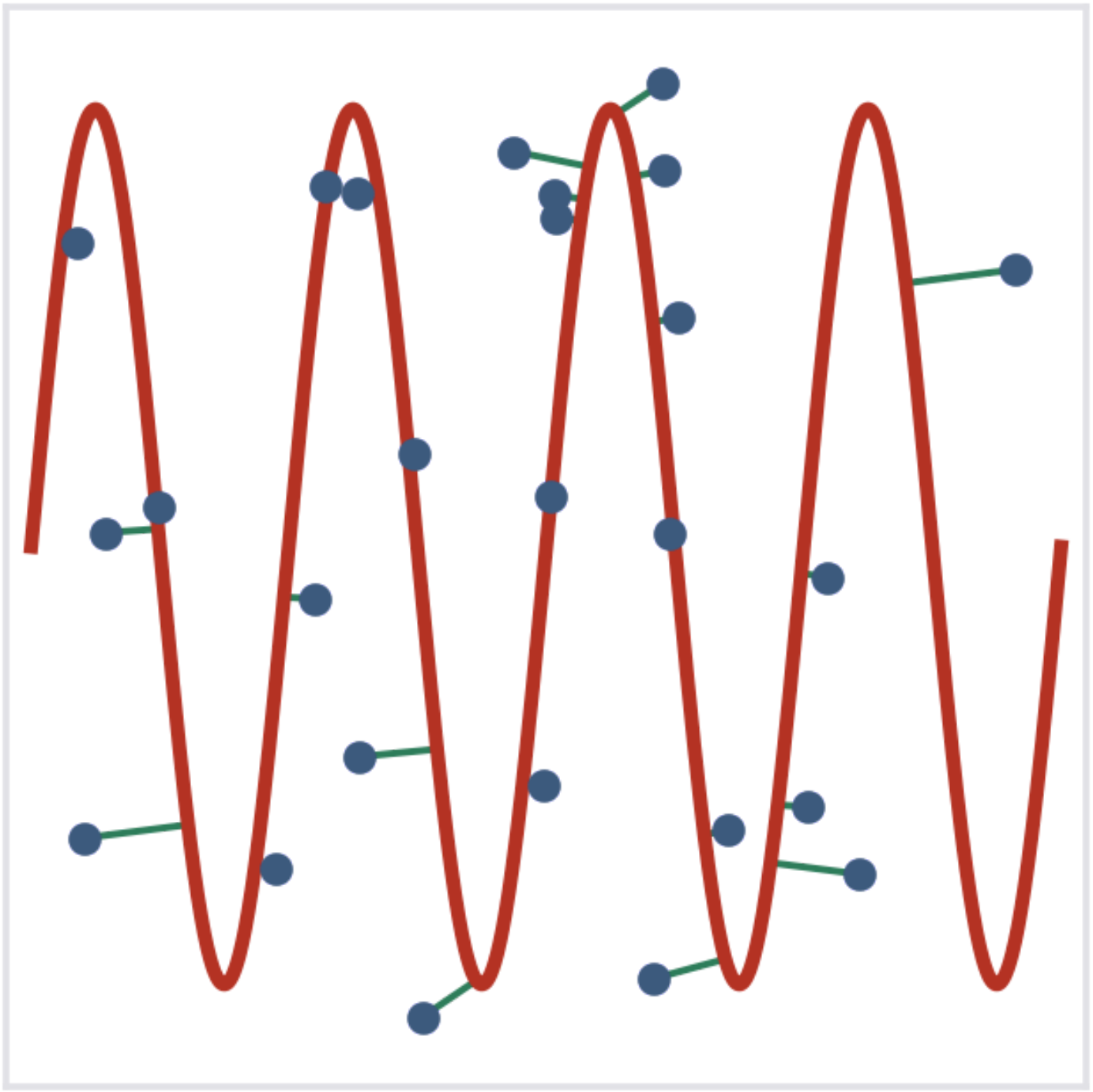
Take a scatter of data points and a curve (a "squiggly line") through the same space. The curve is the function that the network computes. Define the loss as the average distance from each data point to the nearest point on the curve:

```
L(f, D) = (1/n) * sum_i min_x || (x_i, y_i) - (x, f(x)) ||
data D in [-1, 1]^d x R
```

A "more squiggly" curve lies closer to the scatter. I measure squiggliness as the curve length:

```
l(f) = integral of sqrt(1 + ||grad f(x)||^2) dx
```

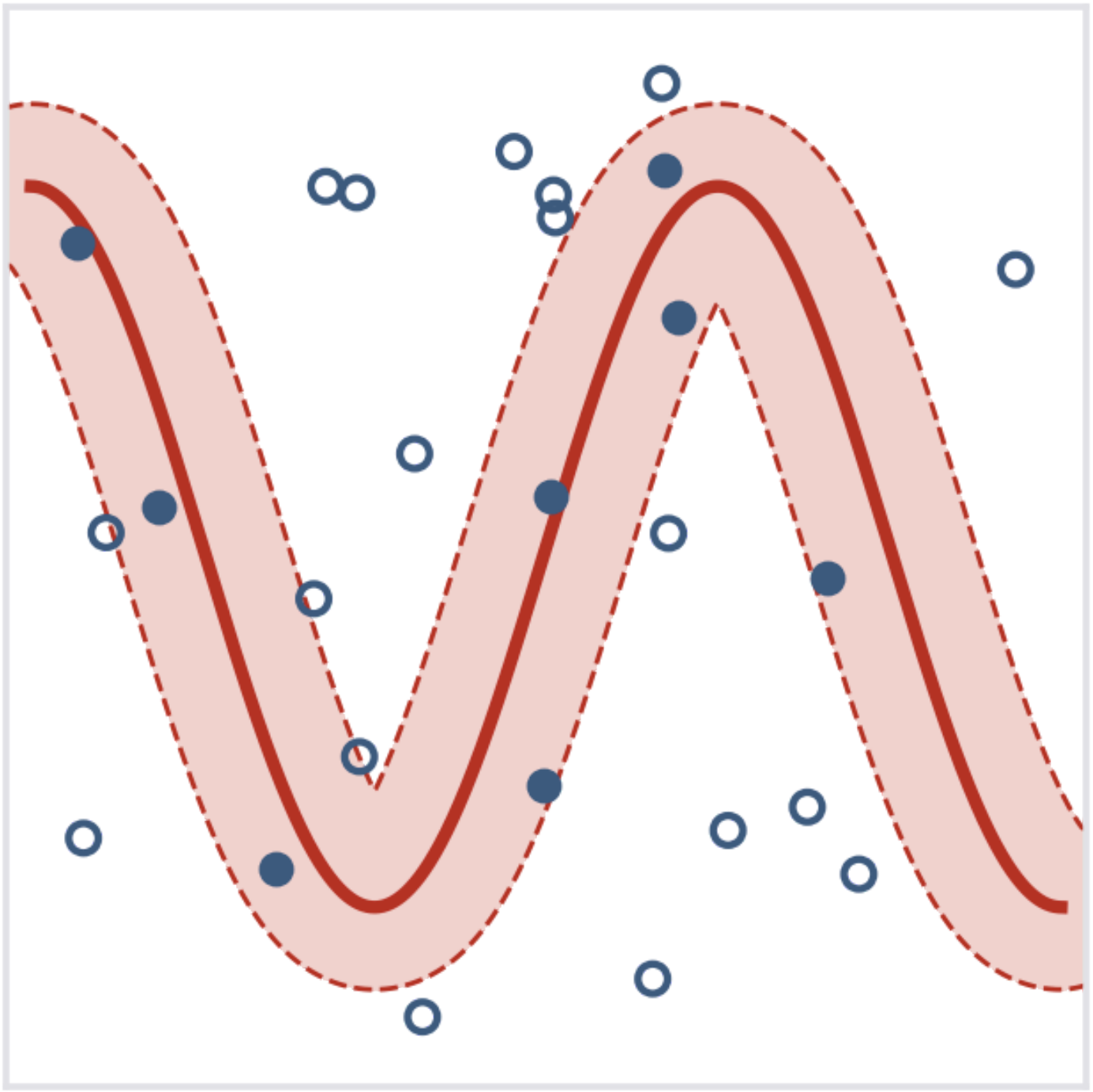




The conjecture (Definition 3.1 in the deck): for two randomly initialised functions with parameter counts |theta_1| < |theta_2|, the expected loss E[L(f_theta_2, D)] is smaller than E[L(f_theta_1, D)]. More parameters, lower expected loss at initialisation.

The tube argument

Draw a tube of radius epsilon around the curve. The volume of the tube grows in proportion to the length: V_covered is about l * k * eps^(d-1). A random point is uncovered with probability V_uncovered / V_total. The original estimate was E[distance] about (V_uncovered / V_total)^(1/d).



The original deck draws two conclusions from this. More length helps, but with diminishing returns. Perfect coverage needs infinite length: there is a wall, but it is at infinity. Also, a curve that folds or loops too tightly wastes length, because its tube overlaps itself.

ReLU networks

ReLU networks are piecewise linear, not polynomials. Where all units are active, the stacked layers collapse to one linear map. A dead unit makes the output constant. So the curve turns only at kinks, and squiggliness is a count of kinks.

Classification as a wind tunnel

In classification, each layer is a place where the wind can change direction. The data points are birds, and the wind carries them to a pigeonhole for each class. A deeper network is a longer tunnel with

finer control. In classification the squiggly line is not visible, because the output is a class.

Autoregression brings the line back

Autoregression is chained classification, so the line returns. The hidden state moves from token to token, and its path is a squiggly line through representation space. Attention lets each step read earlier points on the path.

A correction to the original derivation

In the 2026 review I found a sign error. The original deck took E[dist] about (1 - l*k*eps^(d-1)/V)^(1/d) and concluded diminishing returns from a positive second derivative. For d greater than 1 the second derivative in l is negative (a negative exponent does not flip the sign). So that formula gives increasing returns, the opposite of the slide's conclusion.

Here is a corrected form. Let the tube grow until it fills the space. The graph of f is a surface of area l (for one input dimension, a curve of length l). So the tube is a slab of thickness V / l, and a random point is on average a quarter of that from the surface:

```
E[dist] ~ V / (4 l)
```

This is convex in l. It gives diminishing returns, and zero distance only at infinite length. So the two conclusions stay, for a curve that can reach all the points.

For a 1-D path in D dimensions, for example a hidden-state trajectory, the same argument gives (V / (k l))^(1/(D-1)). This falls very slowly when D is large.

What my experiments found

I ran these in September 2026. The first scripts from 2025 had looked like support for the idea, but their measurement code was wrong. One "distance" was really a count of active neurons. One scaling law was the network output fading to zero. So I started again with one clean experiment.

The experiment uses sine curves with one fixed amplitude and a random phase. Only the frequency changes, so at each x the curve has the same distribution of values for all frequencies. I measured two things against random points: the distance to the nearest point of the curve, and the vertical error |y - f(x)|.

Frequency	Curve length	Nearest-point distance	Vertical error
0.25	2.19	0.6129	0.6672
1	3.96	0.3095	0.6649
4	13.32	0.0928	0.6663

Frequency	Curve length	Nearest-point distance	Vertical error
16	52.31	0.0338	0.6665
64	208.92	0.0207	0.6688

The geometry holds. The nearest-point distance falls by a factor of 30.

The learning claim fails. The vertical error does not move. It stays between 0.6649 and 0.6688. A regression loss charges the vertical error. The wiggles let the curve pass near a point that is to the left or right of it, and training gets nothing for that. The squiggly line is closer to the data in a direction that gradient descent does not measure.

Length is not the mechanism. I stretched a straight line to the same length as the frequency-64 curve (209 units). Its distance was 0.50, against 0.02 for the squiggly curve. That is 24 times worse at the same length. What matters is that the length is spread across the input range.

Random networks agree. For random ReLU networks with 10 inputs and He initialisation, the mean squared error on 200 random points before training was 2.97, 2.98, 3.00, 3.13 and 2.99 for widths 4, 16, 64, 256 and 1024. It is flat. A wider network does not start closer.

The wall is not at infinity. The distance stops falling at a floor. The floor comes from the amplitude: the curve can never reach points outside the band that it sweeps. For my amplitude the floor was 0.017. It does not depend on the number of points. With a band that covers all the data, the distance keeps falling like 1/frequency.

What a second small check found

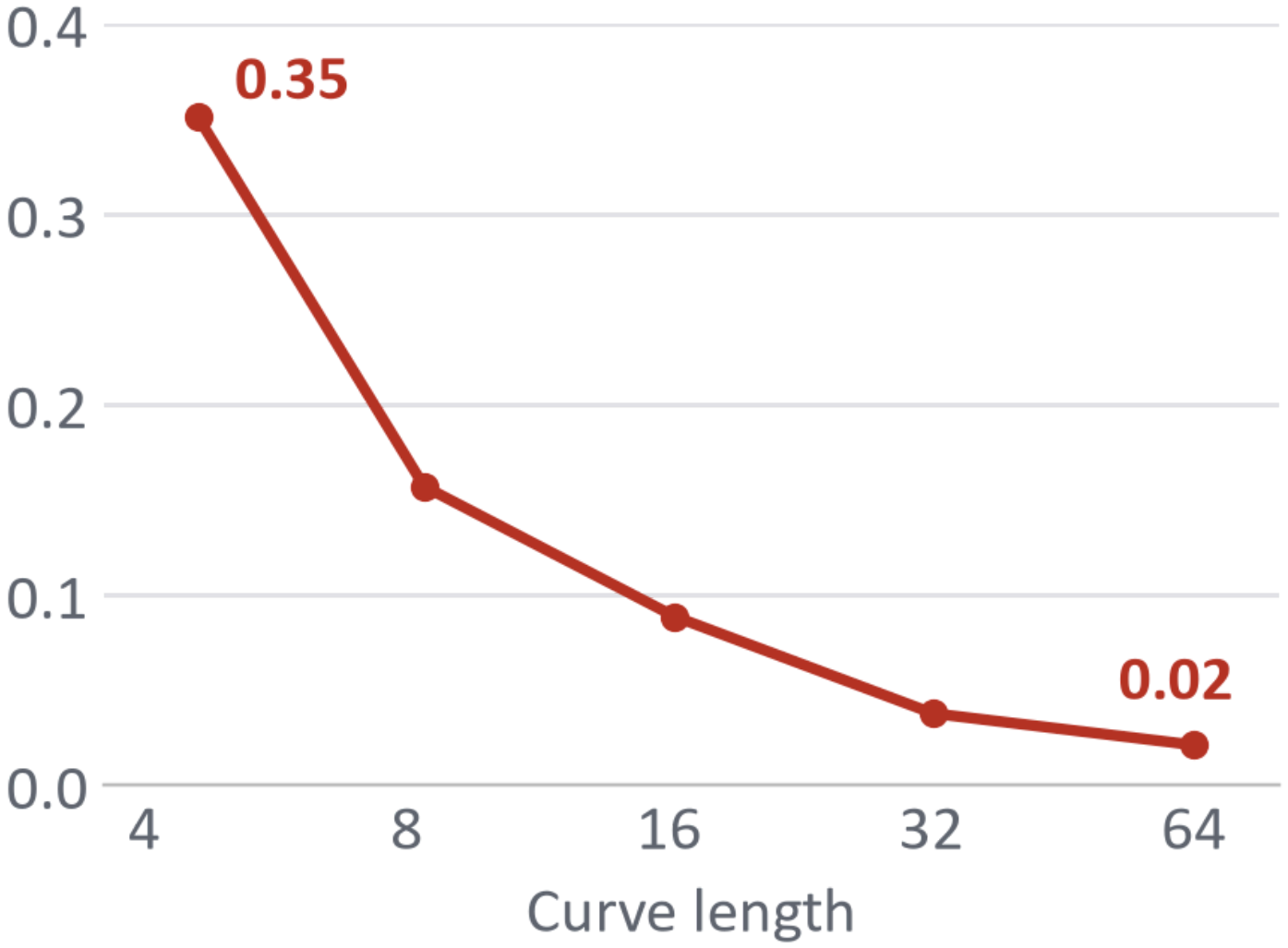
I made this check in November 2026 for the deck revision, before I compared it with the experiments above. It has a 1-D input and no training. It measures only the nearest-point distance, so it tests the geometry and not the learning claim.

Reference curves

First I took sine curves with 1, 2, 4, 8 and 16 periods in a box of side 2, and measured the mean distance to 200 random points.

Periods	Length	Mean distance
1	4.6	0.35
2	8.4	0.16
4	16.2	0.09
8	32.1	0.04
16	64.1	0.02

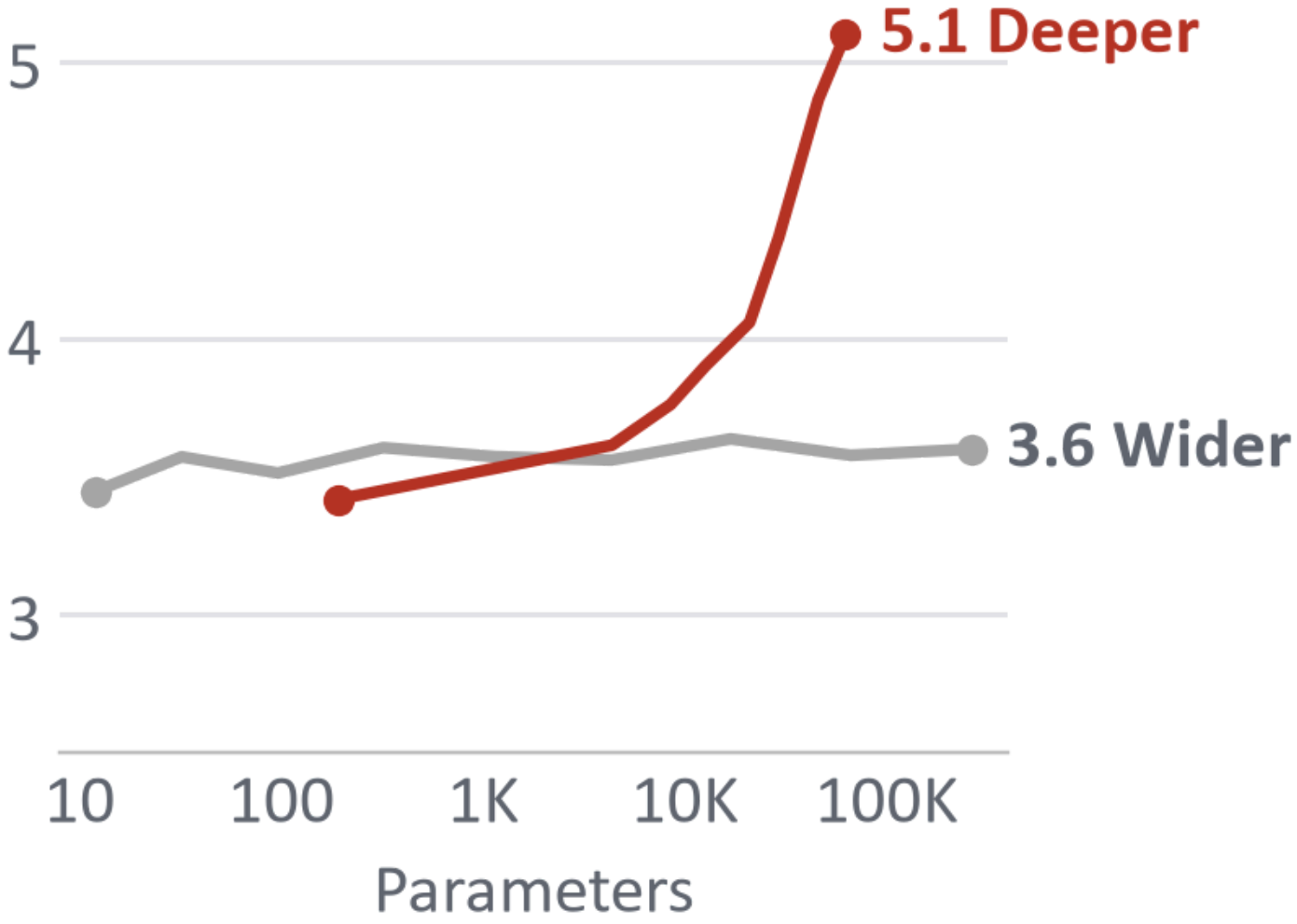
Distance times length stays between 1.2 and 1.6. So the distance falls like 1/length. The geometry part of the idea holds. The corrected formula above predicts a value of 1.0 for this product, and the measured values are 1.2 to 1.6.

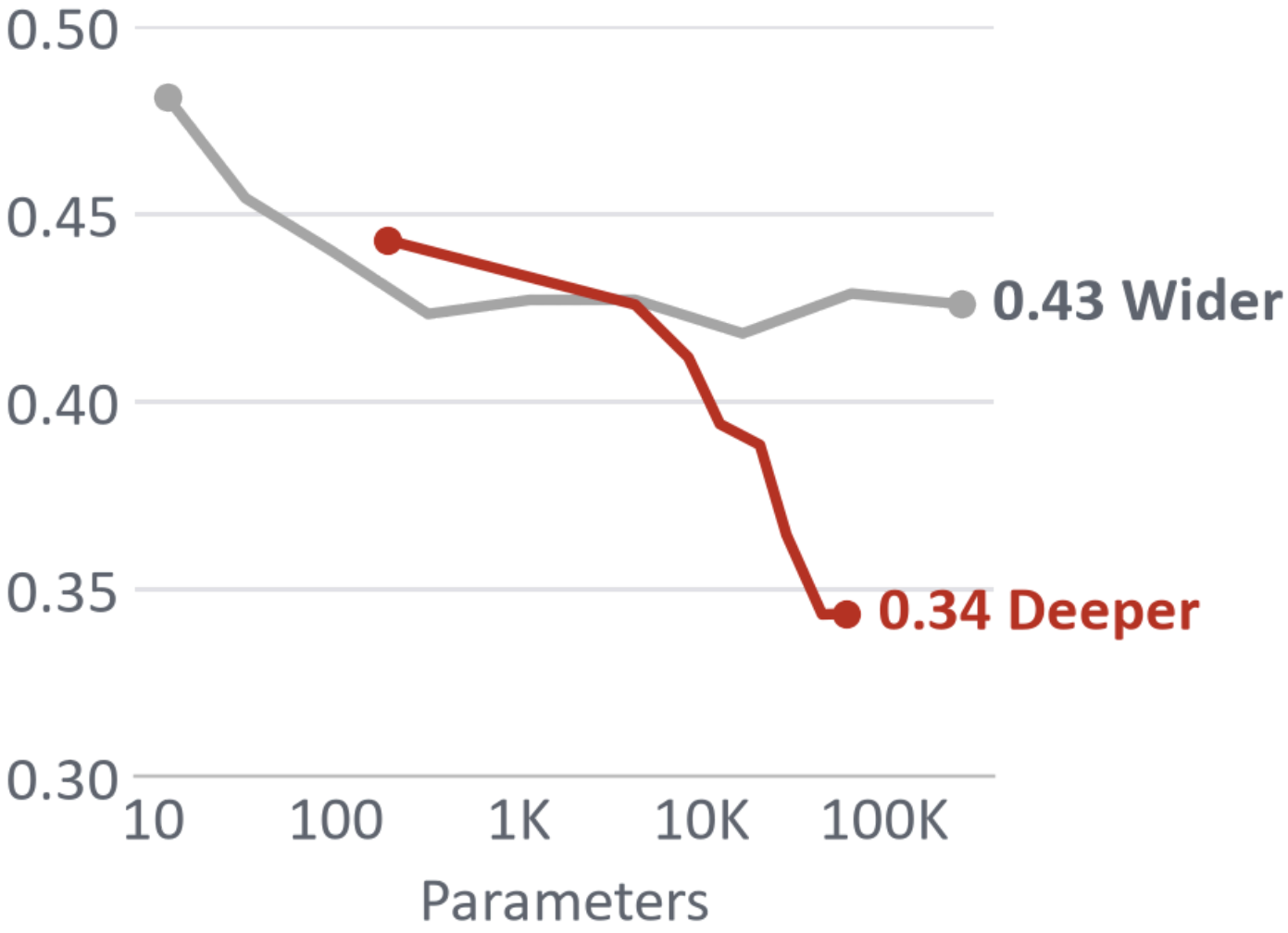


Random ReLU networks

Next I made 300 random ReLU networks of each size, with He-uniform weights. I rescaled the output to span the box.

- Wider networks (2 hidden layers, width 2 to 512): the curve length stays near 3.5 to 3.6 at all widths. The mean distance falls from 0.48 to about 0.42 and then stays flat from width 16.
- Deeper networks (width 64, 1 to 16 hidden layers): the length grows from 3.4 to 5.1. The distance falls from 0.44 to 0.34.
- Over 1,600 pooled random networks, the correlation between length and distance is -0.76.


5.1 Deeper
5
4
3.6 Wider
3
10
100
1K
10K
100K
Parameters



With the PyTorch default initialisation (smaller weights), the random curves are almost flat lines at every size. Their length is 2.0 to 2.1, where a flat line has length 2.0.

My reading: a longer curve is closer to the data in the nearest-point distance, but more parameters do not reliably give a longer curve. Width adds no length in this check. Depth adds some. This agrees with the flat initial error in the experiments above.

What published work says

The summary first: published work backs the geometry (length and coverage) and does not back the parameter claim. That matches my experiments. Here is each paper with my verdict.

Raghu et al., "On the Expressive Power of Deep Neural Networks" (ICML 2017). The length of the image of an input path grows exponentially with depth when the weights are large. Width matters little. This is for the idea, for depth only.

Hanin, Jeong and Rolnick, "Deep ReLU Networks Preserve Expected Length" (ICLR 2022). At standard (He) initialisation the expected length does not grow with depth. It shrinks slightly. This is against the random-initialisation form of the conjecture, if length is the mechanism. My straight-line test says that length was not the mechanism in any case.

Dherin et al., "The Geometric Occam's Razor Implicit in Deep Learning" (2021). It uses the same arc-length measure. SGD pushes the length down, and a trained 1-D ReLU network is near the shortest curve through the data. This is against "longer is better" after training.

Ramanujan et al., "What's Hidden in a Randomly Weighted Neural Network?" (CVPR 2020). A wider random network contains a better subnetwork. This is for the lottery-ticket link. But the gain is in the best subnetwork after a search, not in the average random function.

Larsen, Fort, Becker and Ganguli, "How many degrees of freedom do we need to train deep networks" (ICLR 2022). A random subspace of weight space reaches the low-loss set once it has enough dimensions. This is a coverage argument like mine, but in weight space, not in data space.

Bubeck and Sellke, "A Universal Law of Robustness via Isoperimetry" (NeurIPS 2021). To fit n noisy points smoothly, a network needs many parameters. More parameters allow a smoother fit. So parameters reduce the squiggliness that is necessary. They do not add it.

Chiang et al., "Loss Landscapes are All You Need" (ICLR 2023). Random sampling of weights until the training data are fit ("guess and check") generalises about as well as SGD, and wider networks do better. This supports "size helps without training". But the functions that fit are the simple ones.

Hosseini and Fedorenko (NeurIPS 2023). Trained language models make the token trajectory straighter, and larger models straighten more. This is against the squiggly reading of the token path.

Martinelli, Brea and Gerstner, "Lottery Tickets Do Not Explain Overparameterization. How About Escape Dimensions?" (2026 position paper). It argues that width helps by adding dimensions to escape bad minima, not by better luck at initialisation.

The 2026 revision

The first point is a result of my experiments. The other two are proposals with no test.

1. Spread, not length or parameter count. A straight line of the same length does 24 times worse. Length helps only when it is spread across the input range.
2. Look at reachable curves, not one random curve. A larger network can reach more curves, by training or by a subnetwork search. Measure the coverage of that family.
3. Training spends length. Published results show shorter curves and straighter token paths after training. So squiggliness can be a starting resource, not the end state.

Possible errata

Here are the places where I think the idea or the check can be wrong.

1. The sign error described above.
2. The graph of f is a surface, not a curve, when the input has more than one dimension. The "length" integral is then a surface area, and the tube volume is area times 2 eps.
3. The original distance estimate, (uncovered fraction)^(1/d), is stated with no derivation.

4. The loss is not the training loss. It is the distance to the nearest point of the curve. Regression trains on the vertical error, which is never smaller. My experiments measured this: the effect on the vertical error is zero. This is the erratum that ends the learning claim.
5. The data are assumed to fill the box. Real data lie near a function. A curve that covers all the space also fits noise, so generalisation is not addressed.
6. The links to the lottery ticket hypothesis, double descent and faster learning are claims with no derivation.
7. The experiments and the check are 1-D (the initial-loss table has 10 inputs). They are my own scripts, not a published benchmark.

Next steps

The original question is answered, so these are new questions.

1. Reachability. Does a larger network travel a shorter distance in weight space from its start to a fit? That is "starts closer" in the one space where it can be true.
2. The lower tail. The mean initial error is flat in width. Is the chance of a low initial error also flat?
3. Subnetworks. Compare the best subnetwork of a random network with the full network, on the vertical error.
4. Training. Find out if a trained network keeps, removes or moves its kinks.

Where it stands

The conjecture is true, and true in a direction that training cannot use. A squigglier curve is closer to random points in the nearest-point distance. The vertical error, which training minimises, does not change, and a wider random network does not start with a lower loss. So the picture does not explain why large networks learn fast. The negative result is cleaner than the positive one would have been. I am not claiming priority for any of this. The next job is the reachability question.

Slides: The Squiggly Line Conjecture (2026) and the original 2022 deck

---

What do you think?

0 Responses

 Upvote  Funny  Love  Surprised  Angry  Sad

0 Comments Login ▾

Start the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS

Name

Share Best Newest Oldest

All articles

Causal-Consistency Training: An Idea From 2021 and the Research Since
10 Nov 2026

The Squiggly Line Conjecture: An Idea From 2022 and What the Test Found
10 Nov 2026

Appeasing the Machine Spirit: RLHF in In-Context Learning
03 Nov 2026

Four Tricks at the Edge of Formal Language Theory
15 Aug 2026

Spring-Loaded Dinosaurs: What Penguins, Kiwis, and Armored Core Get Right (and Wrong) About Legs
28 Feb 2026

Passive Velocity Sorting: A Geometric Maxwell's Demon?
14 Dec 2025

AC Gravity and Gravitational Opacity
05 Dec 2025

Theorem: Equivalence of -arccoth and tan Functions
21 Nov 2025

The Photonic Transformer Hypothesis: Rethinking Photosynthesis and Methane Consumption in Plants
19 Nov 2025

The Scalable Quasi-Perpetual Photonic Machine
17 Nov 2025

How to Make a Ball Orbit Itself
08 Nov 2025

The Boron-Nitrogen Catastrophe: A Critical Gap in Beta Decay Verification
28 Oct 2025

X-Ray Data Pipeline: Ultra-High-Speed Communication Through Limestone Tubes
26 Oct 2025

The Universal Approximation Theorem Is Right. You're Using It Wrong.
19 Oct 2025

The Power Law Illusion: A Measurement Artifact Hypothesis
11 Oct 2025

Fractional Gamma Function via Fractional Derivatives
18 Sep 2025

[On the cruel irony of the P-NP problem]
18 Dec 2023

[The Journey from India to Germany: A Guide for IT Professionals]
20 Jun 2023

[How does AutoGPT work under the hood?]
03 May 2023

[Unit Test Recorder - Automatically generate unit tests as you use your application]
25 Jun 2020

[How to migrate from vanilla Kubernetes to Istio service mesh?]
14 Oct 2019